

Practical: 8

23/09/2026

Implementation of Graph and Searching (DFS and BFS).

Aim: Implementation of Graph and Searching (DFS and BFS).

Algorithm:

Algorithm:

Input: vertices V , edges E , List of edges (u, v) , starting vertex $start$

output: DFS & BFS traversal order, execution time for BFS & DFS

Step 1: Read the number of vertices n & edges E

Step 2: Initialize adjacency list adj of size n .

Step 3: For each edge (u, v) entered:

- add v to adjacency list of u
- add u to adjacency list of v

Step 4: Perform DFS traversal

- initialize $visited[]$ as false,
- call recursive function $DFSUtil$ ($start, visited$) which:
- mark node as visited,
- print node,
- recursively visits all ~~un~~unvisited neighbors

Step 5: Perform BFS traversal:

- Initialize $visited[]$ as false
- create queue q
- mark $start$ as visited and enqueue it
- while queue not empty:
- Dequeue node and print it,
- enqueue all unvisited neighbours & mark them visited.

Step 6: measure execution time:

- * Record start & end time for DFS, Compute duration
- * Record start & end time for BFS, Compute duration

Step 7: routes, print DFS traversal order, BFS traversal order.

Program:

```
#include <iostream>

#include <vector>
#include <queue>
#include <chrono>

using namespace std;
using namespace std::chrono;

class Graph
{
    int V;
    vector<vector<int>> adj;

public:
    Graph(int vertices)
    {
        V = vertices;
        adj.resize(V);
    }

    void addEdge(int u, int v)
    {
        adj[u].push_back(v);
        adj[v].push_back(u); // Remove this line for directed graph
    }

    void DFSUtil(int v, vector<bool> &visited)
    {
        visited[v] = true;
        cout << v << " ";
```

```
for(int neighbor : adj[v])
{
    if(!visited[neighbor])
        DFSUtil(neighbor, visited);
}
}
```

```
voidDFS(int start)
{
    vector<bool> visited(V, false);
    DFSUtil(start, visited);
}
```

```
voidBFS(int start)
{
    vector<bool> visited(V, false);
    queue<int> q;
```

```
    visited[start] = true;
    q.push(start);
```

```
    while(!q.empty())
    {
        int node = q.front();
        q.pop();
```

```
        cout<< node << " ";
```

```
        for(int neighbor : adj[node])
        {
            if(!visited[neighbor])
```

```
{
    visited[neighbor] = true;
    q.push(neighbor);
}
}
}
};

int main()
{
    int V, E;

    cout<<"Enter number of vertices: ";
    cin >> V;

    Graph g(V);

    cout<<"Enter number of edges: ";
    cin >> E;

    cout<<"Enter edges (u v):\n";
    for(int i=0; i<E; i++)
    {
        int u, v;
        cin >> u >> v;
        g.addEdge(u,v);
    }

    int start;
    cout<<"Enter starting vertex: ";
```

```
cin >> start;

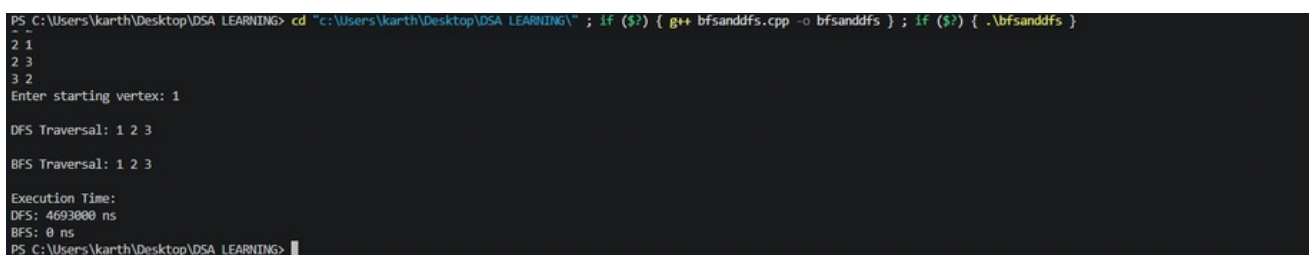
// DFS Time Analysis
auto startDFS = high_resolution_clock::now();
cout << "\nDFS Traversal: ";
g.DFS(start);
auto endDFS = high_resolution_clock::now();

auto dfsTime = duration_cast<nanoseconds>(endDFS - startDFS);

// BFS Time Analysis
auto startBFS = high_resolution_clock::now();
cout << "\n\nBFS Traversal: ";
g.BFS(start);
auto endBFS = high_resolution_clock::now();

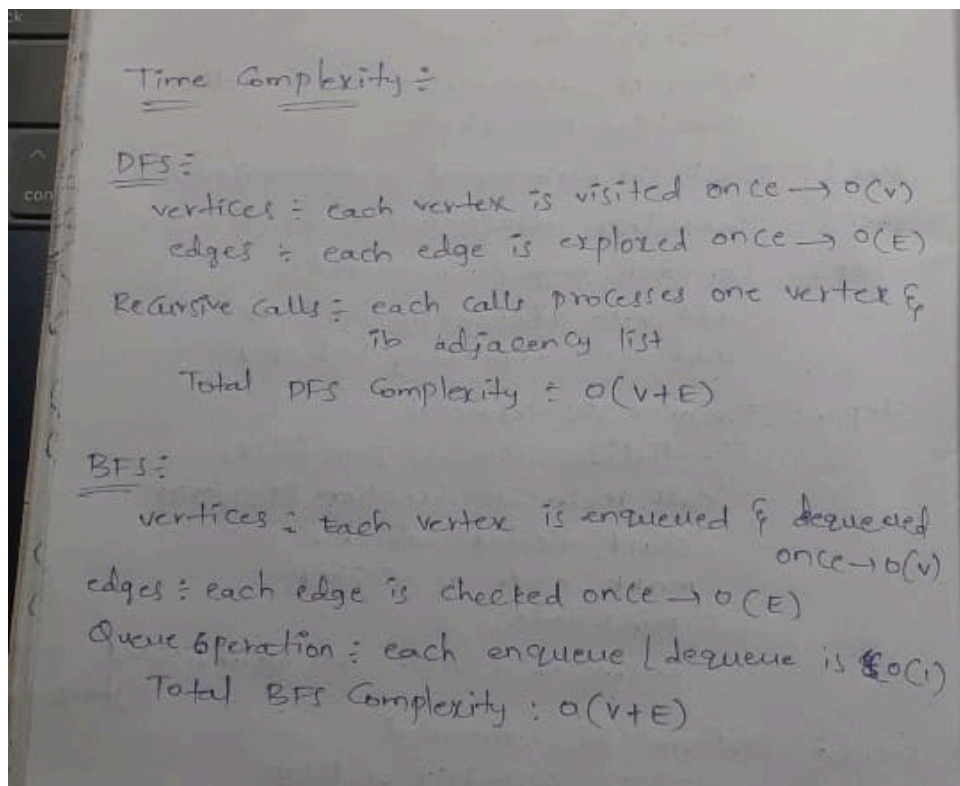
auto bfsTime = duration_cast<nanoseconds>(endBFS - startBFS);

cout << "\n\nExecution Time:";
cout << "\nDFS: " << dfsTime.count() << "
ns"; cout << "\nBFS: " << bfsTime.count() <<
" ns";
return 0;
}
```

Output:

```
PS C:\Users\karth\Desktop\DSA LEARNING> cd "c:\Users\karth\Desktop\DSA LEARNING\" ; if ($?) { g++ bfsanddfs.cpp -o bfsanddfs } ; if ($?) { .\bfsanddfs }
2 1
2 3
3 2
Enter starting vertex: 1
DFS Traversal: 1 2 3
BFS Traversal: 1 2 3
Execution Time:
DFS: 4693000 ns
BFS: 0 ns
PS C:\Users\karth\Desktop\DSA LEARNING>
```

Time complexity:



Conclusion:

Hence, both DFS and BFS are effective graph traversal techniques: DFS explores deeply along each branch before backtracking, while BFS progresses level by level. Their execution times are comparable for small graphs, but BFS is often better for shortest-path problems, whereas DFS is more suited for exhaustive searches.